

UNIVERSIDAD CARLOS III



Sistemas Distribuidos

Práctica. Servicio de envío de mensajes

CURSO 2025 - 2026

XIYA YE → 100522159@alumnos.uc3m.es

Grupo 85, Ingeniería Informática

WENJIE CHEN → 100522255@alumnos.uc3m.es

Grupo 85, Ingeniería Informática

Índice

1. Descripción del código.....	3
1.1. Funciones de utilidad de red (lines.c / lines.h).....	3
1.2. Módulo de almacenamiento (usuarios.c / usuarios.h).....	3
1.3. Servidor de mensajería (servidor.c).....	4
1.3.1. Inicialización.....	4
1.3.2. Operaciones (tratar_cliente).....	4
1.4. Cliente (client.py).....	5
1.5 Servicio web (web_service.py).....	6
1.6 Servicio RPC (registro.x / registro_server.c).....	6
1.7 Cliente de Registro RPC (rpc_client.h / rpc_client.c).....	6
2. Descripción de la forma de compilar y ejecución.....	7
3. Batería de pruebas.....	8
4. Conclusiones.....	11

1. Descripción del código

Para el desarrollo de esta práctica, estructurada siguiendo un modelo cliente-servidor para la mensajería de control, además complementándolo con el envío de mensajes para la transferencia de ficheros. El código del servidor, desarrollado en C, está separado de la lógica de acceso a datos de la lógica de la red. Por otra parte, el cliente se ha implementado en Python.

A continuación, describimos los componentes principales del sistema y sus funciones clave.

1.1. Funciones de utilidad de red (*lines.c* / *lines.h*)

Hemos empezado por crear este módulo para solventar una limitación propia de los sockets TCP, al ser un protocolo orientado a flujo de bytes, esto no nos garantiza que un mensaje enviado se reciba por completo en una sola lectura. Por ello, estas funciones nos proporcionan una capa de abstracción sobre las llamadas al sistema *read* y *write*. Para evitar lecturas fragmentadas, hemos implementado la función *readLine()*, la cual encapsula la lectura byte a byte desde el descriptor del socket hasta localizar un carácter delimitador (como el salto de línea o el carácter nulo). Esta capa de abstracción nos asegura que el servidor procese comandos completos y bien formados. Además, las funciones como *writeLine()* y *write_int()* nos facilitan el envío de datos de manera segura, garantizando que toda la información se codifique como cadenas de caracteres finalizadas con “\0”.

1.2. Módulo de almacenamiento (*usuarios.c* / *usuarios.h*)

Este módulo es el encargado de gestionar los datos del sistema, separándose de la capa de red. En la implementación de esta, hemos decidido utilizar un sistema de ficheros de Linux para el almacenamiento, alojando toda la información en el directorio local *./ALMACEN*.

A nivel estructural, la información de cada usuario se guarda en un fichero de texto independiente con el formato de *nombre_usuario.txt*.

Para el manejo de los mensajes que deben ser almacenados cuando un usuario no está conectado, lo hemos definido con la estructura *MensajePendiente*. Esta estructura tiene los campos *id* (identificador único del mensaje), *remite* (nombre del usuario que envía el mensaje), *mensaje* (contenido del texto) y *fichero* (nombre del fichero adjunto, se almacena la cadena ¡*SIN_ARCHIVO!* cuando no hay ninguno).

El formato de cada fichero *.txt* consta de una cabecera fija de 5 líneas que guarda el estado de conexión (0 para no conectado y 1 para conectado), la IP, el puerto dinámico, el último identificador de mensaje generado por el usuario y el número total de mensajes pendientes. A continuación, en el caso de que existiesen mensajes pendientes, se agrupan en bloques de 4 líneas consecutivas (ID del mensaje, remitente, el propio mensaje y fichero).

A continuación, se explica la lógica de las funciones expuestas en la cabecera *usuarios.h* e implementadas en *usuarios.c*:

- *init_almacen*: Es la función de inicialización. Ejecuta una llamada al sistema (*system*) para forzar el borrado del directorio *./ALMACEN* y su contenido, asegurando un entorno limpio en cada ejecución del servidor. Posteriormente, vuelve a crear el directorio.
- *registrar_usuario*: Esta función empieza por validar los límites de longitud del nombre para posteriormente comprobar si el fichero del usuario ya existe en el directorio. Y si no existe,

crea el fichero y escribe la cabecera inicial con el estado desconectado, IP “0.0.0.0”, puerto y contadores a 0.

- *registrar_usuario*: Verifica la existencia del fichero del usuario, y borra físicamente el archivo. Al eliminarlo, se destruyen también todos sus mensajes pendientes si los tuviera.
- *conectar_usuario*: Abre el fichero del usuario para extraer toda la cabecera y cargar dinámicamente en memoria los mensajes pendientes. Tras verificar que no estaba ya conectado, reescribe el fichero actualizando el estado a 1 e insertando la nueva IP y puerto, volcando de nuevo los mensajes pendientes.
- *desconectar_usuario*: El proceso de este es justo lo contrario de la función anterior. Lee el estado actual, extrae los mensajes pendientes si los hay, y reescribe el fichero del usuario marchando el estado a 0 y vaciando los valores IP y puerto.
- *obtener_siguiete_id_mensaje*: Se encarga de gestionar el identificador secuencial de los mensajes emitidos por un cliente. Abre el archivo del remitente, extrae el valor *ultimo_id_mensaje*, le suma 1 y reescribe el fichero. Finalmente, devuelve el nuevo ID.
- *guardar_mensaje_pendiente*: Lee el fichero del destinatario, incrementa la variable *num_mensajes* en 1, y añade los datos del nuevo mensaje al final del archivo.
- *obtener_mensajes_pendientes*: Lee el número de mensajes en la cabecera y reserva memoria dinámica para los mensajes pendientes. A continuación, rellena línea a línea el fichero y eliminando los saltos de línea.
- *eliminar_mensaje_pendiente*: Esta función se ejecuta cuando recibe un *ACK* exitoso. Lee todos los mensajes, decrementa el contador y reescribe el fichero excluyendo únicamente el mensaje cuyo ID coincida con el procesado.
- *obtener_usuarios_conectados*: Itera sobre cada fichero *.txt* en *./ALMACEN*, extrae el nombre de usuario e invoca a la función *obtener_datos_conexion_usuario()* para filtrar e insertar en una matriz únicamente a aquellos clientes cuyo estado sea conectado (1).

1.3. Servidor de mensajería (*servidor.c*)

En el *servidor.c*, todo el intercambio de información lo realizamos utilizando sockets TCP orientados a conexión, apoyándose en las funciones de abstracción de lectura y escritura que implementados del módulo *lines.c*.

1.3.1. Inicialización

En el hilo principal, el programa configura un socket pasivo, asocia el puerto recibido por argumento mediante *bind()*, y entra en un bucle infinito a la espera de peticiones mediante la llamada bloqueante *accept()*.

Para garantizar la concurrencia, cada vez que un cliente establece conexión, el servidor delega su atención creando un hilo de ejecución independiente utilizando *pthread_create*. Como múltiples hilos operan simultáneamente y pueden necesitar leer o modificar los mismos ficheros, hemos definido una variable global *pthread_mutex_t mutex*, este se encarga de envolver absolutamente todas las llamadas a las funciones de *usuarios.h*, garantizando la exclusión mutua y protegiendo la integridad del directorio *./ALMACEN*.

1.3.2. Operaciones (*tratar_cliente*)

El ciclo de vida de cada hilo lo dirige la rutina *tratar_cliente*, la cual extrae el comando de la operación mediante *readLine()* y deriva la ejecución al bloque correspondiente.

- *REGISTER, UNREGISTER, DISCONNECT*: Estas rutinas actúan como intermediarias. leen el nombre del usuario desde el flujo de red, invocan a la función correspondiente, y envían un byte binario de vuelta al cliente.
- *CONNECT*: Aquí, el servidor lee el nombre del usuario y el puerto de escucha dinámico que el cliente ha abierto, extrae la dirección IP real del emisor sobre el propio socket. Tras actualizar el estado, la rutina verifica si el usuario tiene mensajes pendientes. De ser así, el servidor abre un nuevo socket TCP contra la IP y puerto del usuario, y le entrega todos los mensajes (usando los comandos *SEND_MESSAGE* o *SEND_MESSAGE_ATTACH*). Por cada entrega exitosa, el servidor abre una tercera conexión hacia el remitente original para notificar el recibo (*SEND_MESS_ACK* o *SEND_MESSAGE_ATTACH_ACK*) y elimina el mensaje de la base de datos.
- *SEND* y *SENDATTACH*: Al procesar un envío, el servidor extrae las cadenas con el remitente, el destinatario, el propio mensaje y, opcionalmente, el nombre del fichero. En primer lugar, genera un identificador secuencial y almacena el mensaje en el fichero del destinatario. Tras confirmar la recepción al emisor, el servidor verifica si el destinatario está en línea. Si lo está, abre un socket para realizar la entrega inmediata, y acto seguido, elimina el mensaje. Si durante la llamada a *connect()* ocurre un fallo, el sistema asume que el cliente ha sufrido una descoxi3n abrupta, lo marca como desconectado y mantiene el mensaje.
- *USERS*: El servidor consulta la capa de datos para obtener los clientes en línea y formatea la respuesta (*usuario :: IP :: puerto*). Esta cadena es la que permite a nuestro cliente Python construir su diccionario de red para lanzar la orden GETFILE.

1.4. Cliente (*client.py*)

El cliente lo hemos desarrollado en Python siguiendo un diseño orientado a objetos.

- *_tratar_hilo()*: Esta es la función central de la asincronía del cliente. Se ejecuta en un hilo secundario independiente y contiene un bucle infinito que acepta conexiones entrantes. Su tarea es decodificar la operación recibida y actuar en consecuencia: si recibe notificaciones del servidor (como *SEND_MESSAGE*, *SEND_MESS_ACK*), las imprime por consola. Además, en el caso de que recibiera la petición *GET_FILE* de otro usuario, abre el archivo local y transmite su contenido a través del socket.
- *connect(user)*: Gestiona la conexión al sistema. Crea un socket de escucha vinculando al puerto 0, lanza el hilo ejecutando la función *_tratar_hilo*. Finalmente, establece conexión con el servidor principal.
- *disconnect(user)*: Se encarga de cerrar la sesión del usuario. Envía el comando de desconexión al servidor, y cierra el socket de escucha.
- *users(no_imprimir)*: Solicita la lista de usuarios conectados. Esta función lee la respuesta del servidor (en formato *usuario :: IP :: puerto*) y actualiza el atributo *_usuarios_conectados*. El parámetro *no_imprimir* nos permite ejecutar esta función en segundo plano para refrescar las direcciones sin imprimir en la terminal.
- *send(user, message)* y *sendAttach(user, file, message)*: Gestionan el envío de mensajes normales y con archivos adjuntos. Ambas abren una conexión TCP, envían la estructura del comando y esperan a recibir el byte de retiro junto con el identificador asignado al mensaje.
- *getfile(user, file_name, local_file_name)*: Primero busca la IP y el puerto del usuario en el diccionario *_usuarios_conectados* (invocando a *users()* si no lo encuentra). Una vez obtenida la dirección, abre una conexión TCP contra el cliente remoto, le envía el comando *GET_FILE* y entra en un bucle de lectura. Los fragmentos recibidos se escriben en el disco local abriendo el fichero destino.

1.5 Servicio web (web_service.py)

Este módulo implementa un microservicio RESTful mediante el framework **Flask** en Python (puerto 8000), encargado de la normalización de mensajes para optimizar el almacenamiento del servidor principal.

La lógica del servicio se centraliza en el endpoint `/remove_nwhitespaces` (método **POST**):

- **Normalización:** El servicio recibe un objeto JSON con el campo `message`. Utiliza el método `.split()` para fragmentar la cadena eliminando cualquier secuencia de espacios en blanco, tabulaciones o saltos de línea, y posteriormente reconstruye el texto con `.join()` empleando un único espacio como separador.
- **Gestión de Respuestas:** Tras el procesamiento, devuelve la cadena normalizada con un código de estado **202**. En caso de formatos inválidos o errores de lectura, el sistema captura la excepción y retorna un error **415**.
- **Aislamiento:** Al ejecutarse como un servicio independiente en **127.0.0.1**, permite delegar la manipulación de strings fuera del servidor C, garantizando que errores en el preprocesamiento no afecten a la disponibilidad del servicio de mensajería.

1.6 Servicio RPC (*registro.x* / *registro_server.c*)

Este módulo implementa un servicio de monitorización remoto basado en **RPC** para centralizar las trazas de actividad en un proceso independiente.

- **Definición (*registro.x*):** Define el programa `LOGSRPC_PROG` y la estructura `operacion_usuario` (nombre, operación y fichero). Gracias a **rpcgen**, se generan automáticamente los filtros **XDR** para la serialización de datos entre el servidor de mensajería y el de logs.
- **Implementación (*registro_server.c*):** La función `imprimir_log_1_svc` discrimina el formato de salida: si existe un nombre de fichero (operación `SENDATTACH`), lo incluye en la traza; de lo contrario, imprime solo el usuario y la operación. Utiliza `fflush(stdout)` para asegurar la visualización inmediata de los registros en consola.

1.7 Cliente de Registro RPC (*rpc_client.h* / *rpc_client.c*)

Para desacoplar el protocolo RPC de la lógica principal del servidor de mensajería, se ha desarrollado una capa de abstracción que encapsula toda la complejidad de las llamadas remotas.

- **Abstracción (*rpc_client.h*):** Define la función `registrar_operacion`, permitiendo invocar el registro de actividad como si fuese una función local de C, ocultando la complejidad de los handles de cliente y filtros XDR.
- **Implementación (*rpc_client.c*):** Gestiona la conexión mediante `clnt_create`. Se ha diseñado con **tolerancia a fallos**: si el servidor de logs no está activo, la función retorna un error pero no bloquea al servidor principal, permitiendo que la mensajería continúe.
- **Gestión de Recursos:** La función asegura la liberación de recursos mediante `clnt_destroy` tras cada llamada, evitando fugas de memoria o descriptores abiertos durante la ejecución prolongada del servidor.

2. Descripción de la forma de compilar y ejecución

En este apartado, describiremos los pasos necesarios para generar los ejecutables del sistema y el procedimiento para ejecutarlos.

El servidor central la hemos desarrollado en C, por lo que requiere la compilación de varios módulos (*servidor.c*, *usuarios.c* y *lines.c*). Para ello, hemos creado un archivo Makefile.

- Comando de compilación: `make`

(Al ejecutar este comando, tenemos en cuenta que salen algunos warnings, pero provienen todos de los archivos generados automáticamente por el comando *rpcgen*.)

Una vez generado el binario, lanzamos el servicio web en la Terminal 1:

- Lanzamos el servidor web con: `python3 web_service.py`

A continuación, lanzamos el servidor RPC en la Terminal 2:

- Lanzamos el rpc con: `./rpc_server`

Con esto, en la Terminal 3, podemos arrancar el servidor principal, indicando la IP del servidor RPC mediante la variable de entorno *LOG_RPC_IP*:

- `export LOG_RPC_IP=<IP>`
- `./server -p <PUERTO>`

Finalmente, en otra terminal, todos usuarios pueden conectarse al sistema ejecutando el cliente, como se trata de un script interpretado, este no requiere de una fase de compilación previa, pero sí necesita un entorno de ejecución Python 3. Le debemos proporcionar la IP y el puerto.

- Lanzamos el cliente con: `python3 client.py -s <IP> -p <PUERTO>`

3. Batería de pruebas

Prueba	Entrada	Resultado Esperado	Obtenido
Registro: Un cliente solicita registrarse.	c> REGISTER mari	c> REGISTER OK	Éxito
Registro duplicado: Un cliente solicita registrarse con un nombre que ya existe.	c> REGISTER mari	c> REGISTER FAIL, USER ALREADY EXISTS	Éxito
Baja: El cliente previamente registrado, solicita darse de baja.	c> UNREGISTER mari	c> UNREGISTER OK	Éxito
Baja de usuario inexistente: El cliente intenta darse de baja sin estar registrado.	c> UNREGISTER carmen	c> USER DOES NOT EXIST	Éxito
Conexión: Un cliente (registrado) intenta conectarse.	c> CONNECT mari	c> CONNECT OK	Éxito
Conexión de usuario inexistente: Un cliente (no registrado) intenta conectarse.	c> CONNECT carmen	c> CONNECT FAIL, USER DOES NOT EXIST	Éxito
Conexión duplicada: Un cliente ya conectado vuelve a intentar conectarse.	c> CONNECT mari	c> USER ALREADY CONNECTED	Éxito
Lista de usuarios conectados.	c> USERS	c> CONNECTED USERS (1 users connected) OK mari	Éxito
Lista de usuarios, sin estar conectado.	c> USERS	c> CONNECTED USERS FAIL, USER IS NOT CONNECTED	Éxito
Desconexión: Un cliente intenta desconectarse.	c> DISCONNECT mari	c> DISCONNECT OK	Éxito
Desconexión sin estar conectado.	c> DISCONNECT carmen	c> DISCONNECT FAIL, USER DOES NOT EXIST	Éxito

Prueba	Entrada	Resultado Esperado	Obtenido
Envío a usuario inexistente: Un cliente intenta enviar un mensaje a otro que no esta registrado.	c> SEND sancho Hola! Que tal?	c> SEND FAIL, USER DOES NOT EXIST	Éxito
Envío a un usuario desconectado. Más tarde, cuando se conecta el destinatario.	c> SEND carmen Hola c> CONNECT carmen	SEND MESSAGE 2 OK c> CONNECT OK c> c> MESSAGE 2 FROM mari Hola END	Éxito
Envío a usuario conectado.	c> SEND carmen Como estas	c> SEND OK - MESSAGE 2 c> MESSAGE 2 FROM mari Como estas END	Éxito
Envío con fichero adjunto.	c> SENDATTACH carmen mira carmen foto.txt	c> SENDATTACH OK - MESSAGE 1 c> MESSAGE 1 FROM mari mira carmen END FILE: foto.txt	Éxito
Descarga del fichero enviado.	c> GETFILE mari foto.txt mi-foto.txt	c> FILE mi-foto.txt DOWNLOADED OK	Éxito
Descarga del fichero cuando el cliente está desconectado.	c> GETFILE mari foto.txt mi-foto.txt	c> FILE TRANSFER FAILED, user not connected	Éxito
Distintos remitentes mandan mensajes a la vez a un mismo usuario: Para ello, hemos creado un script en Python que ejecute registre 3 usuarios y los conecten. Después cada usuario mandará varios mensajes a otro usuario previamente	c> REGISTER Jorge c> REGISTER OK c> CONNECT Jorge c> CONNECT OK python3 test_concurrencia.py	c> MESSAGE 1 FROM Jose Mensaje 1 de Jose END c> c> MESSAGE 1 FROM Lucia Mensaje 1 de Lucia END c> c> MESSAGE 1 FROM Diana Mensaje 1 de Diana END c>	Éxito

registrado y conectado. En este caso lo probamos con 5 mensajes, pero para probar mejor la concurrencia lo probamos con 50 mensajes cada uno.		<pre>c> MESSAGE 2 FROM Jose Mensaje 2 de Jose END c> c> MESSAGE 2 FROM Lucia Mensaje 2 de Lucia END c> c> MESSAGE 2 FROM Diana Mensaje 2 de Diana END c> c> MESSAGE 3 FROM Jose Mensaje 3 de Jose END c> c> MESSAGE 3 FROM Lucia Mensaje 3 de Lucia END c> c> MESSAGE 3 FROM Diana Mensaje 3 de Diana END c> c> MESSAGE 4 FROM Lucia Mensaje 4 de Lucia END c> c> MESSAGE 4 FROM Diana Mensaje 4 de Diana END c> c> MESSAGE 4 FROM Jose Mensaje 4 de Jose END c> c> MESSAGE 5 FROM Jose Mensaje 5 de Jose END c> c> MESSAGE 5 FROM Diana Mensaje 5 de Diana END c> c> MESSAGE 5 FROM Lucia Mensaje 5 de Lucia END</pre>	
---	--	--	--

4. Conclusiones

El desarrollo de esta práctica sobre un servicio de envío de mensajes nos ha permitido consolidar de forma práctica los conceptos teóricos de esta asignatura de Sistemas Distribuidos.

Algunos problemas que hemos tenido a la hora de implementar han sido sobre todo algunos pequeños detalles de los que no nos dábamos cuenta que pasaban. Por ejemplo, al usar *fgets()* para leer datos como la IP o el puerto de los archivos *.txt*, esta función captura también el carácter de salto de línea. Y al volver a guardar los datos usando *fprintf()*, nos generaba líneas en blanco de más. Para solucionar eso, descubrimos la función *strcspn* que podíamos usar para eliminar ese salto de línea.

Otro de los problemas que nos hemos encontrado ha sido que a la hora de probar los envíos de mensajes, cuando el cliente recibía los mensajes asíncronos, estos no aparecían en la pantalla. Nos dimos cuenta que se debía a que en la función *print()* de Python con el parámetro *end=""* retenía el texto en el buffer de salida estándar. Pero lo pudimos solucionar añadiendo el parámetro *flush=True* para forzar el vaciado del buffer e imprimir el texto en pantalla.

Además, al principio para el comando USERS, usábamos *strcon(nombre, '.txt')* para extraer el nombre del usuario de los archivos en el servidor. Descubrimos que esto truncaba cualquier nombre de usuario que contuviera letras como la "t" o la "x". Para ello, lo sustituimos por *strrchr(nombre, '.')* para buscar de forma más segura el último punto de la cadena, y así poder aislar el nombre del usuario.

A nivel personal, el desarrollo de este sistema distribuido nos ha resultado bastante complejo debido a la dificultad que supone depurar aplicaciones concurrentes. Gran parte del tiempo lo invertiremos en solucionar fallos sutiles. Aunque también consideramos que el resultado final conseguido está bastante completo y ya que quisimos implementar tanto la Parte 1 como la Parte 2.